

Manual de Linux e Shell Scripting

Do básico ao avançado — do terminal ao kernel, com Kali Linux como referência

Vitor Mattos

01/08/2026

Índice

Apresentação	1
Para quem é este manual	1
Como o manual está organizado	1
Como cada capítulo funciona	1
I. Volume 1 — Primeiros Passos: Linux e Kali	3
1. O que é Linux: kernel, distribuições e o modelo de software livre	5
1.1. O que a máquina responde no primeiro minuto	5
1.2. O kernel: o Linux no sentido estrito	6
1.3. De onde veio: Unix, GNU e 1991	7
1.4. O que é, então, uma distribuição	8
1.5. Famílias: o mapa das distribuições	9
1.6. O modelo de software livre	10
1.7. O que muda no seu dia a dia	10
1.8. No Kali	10
1.9. Laboratório	11
1.10. Resumo	13
2. Por que Kali Linux: propósito, filosofia e quando não usá-lo	15
3. Instalando o Kali: máquina virtual, bare metal, WSL e live USB	17
4. Primeiro contato: sessão, ambiente Xfce e organização do sistema	19
5. A estrutura de diretórios do Linux (FHS)	21
6. Anatomia de um comando: shell, argumentos, opções e sintaxe	23
7. Obtendo ajuda: man, info, tldr e a documentação do Kali	25
II. Volume 2 — O Terminal e o Shell	27
8. Emulador de terminal e shell: bash, zsh e o padrão do Kali	29
9. Navegação: pwd, cd, ls e caminhos absolutos e relativos	31
10. Manipulando arquivos e diretórios: cp, mv, rm, mkdir	33
11. Visualizando conteúdo: cat, less, head, tail e more	35
12. Redirecionamento e pipes: stdin, stdout e stderr	37
13. Curingas, globbing e expansão de chaves	39

14. Histórico, atalhos de teclado e produtividade no terminal	41
 III. Volume 3 — Arquivos e o Sistema de Arquivos	 43
15. Tipos de arquivo, inodes e a árvore de diretórios	45
16. Links simbólicos e hard links	47
17. Localizando arquivos: find, locate, which e type	49
18. Compactação e arquivamento: tar, gzip, xz e zip	51
19. Montagem de dispositivos e o comando mount	53
20. Permissões de arquivo: leitura, escrita e execução	55
 IV. Volume 4 — Usuários, Grupos e Permissões	 57
21. Usuários, grupos e o arquivo /etc/passwd	59
22. O root, sudo e a elevação de privilégios	61
23. Permissões especiais: SUID, SGID e sticky bit	63
24. ACLs e atributos estendidos de arquivo	65
25. Gerenciamento de usuários, senhas e o PAM	67
26. Trabalhando como root no Kali com segurança	69
 V. Volume 5 — Processos e Controle de Tarefas	 71
27. O que é um processo: PID, estados e a árvore de processos	73
28. Monitoramento: ps, top e htop	75
29. Sinais e controle: kill, killall, nice e prioridades	77
30. Jobs, primeiro e segundo plano, nohup e disown	79
31. Multiplexação de terminal: tmux e screen	81
32. Recursos do sistema: memória, CPU, uptime e limites	83
 VI. Volume 6 — Gerenciamento de Pacotes	 85
33. APT e o modelo Debian: repositórios e o Kali rolling	87
34. Instalando, removendo e atualizando pacotes com apt	89
35. dpkg, arquivos .deb e resolução de dependências	91

36.Repositórios do Kali, metapacotes e kali-tweaks	93
37.Compilando a partir do código-fonte	95
38.Alternativas: pipx, snap, flatpak e Appliance	97
VII.Volume 7 — Boot, systemd e Serviços	99
39.O processo de inicialização: UEFI, GRUB e o kernel	101
40.systemd: units, targets e o modelo de serviços	103
41.Gerenciando serviços com systemctl	105
42.Logs com journald e o syslog tradicional	107
43.Agendamento: cron, at e timers do systemd	109
44.Targets, modo de recuperação e resolução de problemas de boot	111
VIII.Volume 8 — Armazenamento e Sistemas de Arquivos	113
45.Discos, partições e a nomenclatura de dispositivos	115
46.Particionamento com fdisk, parted e gdisk	117
47.Sistemas de arquivos: ext4, Btrfs, XFS e formatação	119
48.LVM: volumes lógicos flexíveis	121
49.RAID por software com mdadm	123
50.Criptografia de disco com LUKS e a persistência no Kali	125
IX. Volume 9 — Fundamentos de Shell Scripting	127
51.Do comando ao script: o primeiro script Bash	129
52.Variáveis, o ambiente e o escopo de exportação	131
53.Aspas, expansões e substituição de comandos	133
54.Entrada e saída: read, echo e printf	135
55.Condicionais: test, colchetes duplos e if	137
56.Códigos de saída e o encadeamento de comandos	139
57.Boas práticas: shebang, permissões e portabilidade	141

X. Volume 10 — Scripting Intermediário	143
58.Laços: for, while e until	145
59.Funções, retorno e escopo de variáveis	147
60.Arrays indexados e associativos	149
61.Parâmetros posicionais e getopts	151
62.Expansão de parâmetros avançada	153
63.Aritmética e manipulação de números no shell	155
XI. Volume 11 — Processamento de Texto	157
64.Expressões regulares: fundamentos e os diferentes sabores	159
65.grep e a busca de padrões	161
66.sed: o editor de fluxo	163
67.awk: processamento por campos e registros	165
68.cut, sort, uniq, tr e a caixa de ferramentas de texto	167
69.Dados estruturados na linha de comando: jq, JSON e CSV	169
70.Juntando tudo: pipelines de processamento de dados	171
XII. Volume 12 — Scripting Avançado e Robusto	173
71.Tratamento de erros: set -euo pipefail e traps	175
72.Depuração de scripts: set -x e ShellCheck	177
73.Manipulando arquivos, descritores e processos em scripts	179
74.Subshells, agrupamento e here-documents	181
75.Sinais, processos filhos e execução em paralelo	183
76.Interação com o usuário: menus, cores e caixas de diálogo	185
77.Estruturando scripts grandes e bibliotecas de funções	187
XIII. Volume 13 — Redes no Linux	189
78.Fundamentos: interfaces, endereçamento IP e a pilha de rede	191
79.Configuração de rede: ip, NetworkManager e arquivos	193
80.Diagnóstico: ping, traceroute, ss, dig e mtr	195

81. Acesso e transferência remota: SSH, scp e rsync	197
82. HTTP na linha de comando: curl, wget e netcat	199
83. Firewall no Linux: nftables e iptables	201
84. Captura e análise de tráfego: tcpdump e tshark	203
XIV. Volume 14 — Segurança e o Ambiente Kali	205
85. Hardening do Linux: superfície de ataque e princípios	207
86. Gerenciamento de segredos: chaves SSH e GPG	209
87. Capabilities, isolamento e menor privilégio	211
88. Confinamento: AppArmor, SELinux e sandboxing	213
89. Auditoria, integridade de arquivos e detecção de alterações	215
90. O ecossistema de ferramentas do Kali: categorias e fluxos	217
91. Anonimato e privacidade: proxychains, Tor e VPN	219
XV. Volume 15 — Automação e Produtividade	221
92. Personalizando o shell: .bashrc, .zshrc, aliases e funções	223
93. Zsh, plugins e o prompt do Kali	225
94. Automatizando tarefas com cron e scripts agendados	227
95. Controle de versão com Git na linha de comando	229
96. Editores no terminal: vim e nano	231
97. Dotfiles e ambientes reproduzíveis	233
XVI. Volume 16 — Kernel, Contêineres e Virtualização	235
98. O kernel Linux: módulos, /proc e /sys	237
99. Contêineres por dentro: namespaces, cgroups e Docker	239
100. Virtualização: KVM, QEMU e o laboratório de estudos	241
101. Compilação, toolchains e o ecossistema de desenvolvimento	243
102. Observabilidade e desempenho: strace, ltrace e perf	245
103. Para onde ir depois: certificações, comunidade e prática contínua	247
Referências	249

Apresentação

Este é um manual de **Linux e shell scripting** escrito em português do Brasil, pensado para levar o leitor do primeiro comando no terminal até a administração avançada do sistema, a automação com scripts robustos e os fundamentos do kernel, contêineres e virtualização.

A distribuição de referência é o **Kali Linux**. Todos os comandos, caminhos de pacote e convenções foram adaptados ao Kali — que é, por baixo, um Debian *rolling release*. Onde o Kali difere de um Debian ou Ubuntu comum (o repositório rolling, o usuário padrão, os metapacotes de ferramentas, o `kali-tweaks`, a persistência em live USB), o texto aponta a diferença explicitamente. Quem usa outra distribuição derivada de Debian encontrará quase tudo aplicável; onde a família RHEL (`dnf`, `firewalld`, SELinux) diverge de forma relevante, há uma nota.

Para quem é este manual

Para quem quer aprender Linux a sério: o estudante que acabou de instalar o Kali numa máquina virtual, o profissional de infraestrutura que administra servidores e redes, e quem vem da área de segurança e precisa dominar o sistema operacional que está sob todas as ferramentas. Não se pressupõe conhecimento prévio de Linux — o Volume 1 começa do zero — mas o manual não para no básico: os últimos volumes tratam de LVM, `nftables`, namespaces e `cgroups`.

Como o manual está organizado

O conteúdo avança em cinco fases, do concreto ao abstrato:

- **Fase 1 — Fundamentos do Linux.** O terminal, o sistema de arquivos, usuários e permissões. A base sobre a qual todo o resto se apoia.
- **Fase 2 — Administração do Sistema.** Processos, pacotes, `systemd`, serviços e armazenamento.
- **Fase 3 — Shell Scripting.** Do primeiro script Bash ao processamento de texto com `sed` e `awk` e ao scripting robusto com tratamento de erros e depuração.
- **Fase 4 — Redes, Segurança e Automação.** Redes no Linux, hardening, o ecossistema de ferramentas do Kali e a automação do dia a dia.
- **Fase 5 — Fronteira e Aprofundamento.** Kernel, contêineres, virtualização e para onde seguir depois.

Como cada capítulo funciona

Cada capítulo abre por um problema concreto, desenvolve o assunto com exemplos executáveis e comentados, e fecha com duas seções fixas: **No Kali**, que destaca o que muda especificamente nessa distribuição, e **Laboratório**, um roteiro prático para reproduzir o conteúdo no seu próprio ambiente. Todo comando mostrado é para ser digitado — a proposta é aprender fazendo.

Apresentação

Aviso. Os capítulos de segurança e das ferramentas do Kali têm caráter educacional e defensivo. Técnicas e ferramentas ofensivas aparecem para que o leitor entenda, detecte e mitigue — e devem ser usadas apenas em ambientes próprios ou com autorização explícita. Usar essas ferramentas contra sistemas de terceiros sem permissão é crime no Brasil (Lei 12.737/2012).

Bom proveito — e mantenha um terminal aberto ao lado enquanto lê.

Parte I.

**Volume 1 — Primeiros Passos: Linux e
Kali**

1. O que é Linux: kernel, distribuições e o modelo de software livre

Três e vinte da manhã. O monitoramento avisa que o servidor que concentra os coletores de tráfego de doze torres parou de responder. Você entra por SSH pelo link de gerência e o prompt abre — a máquina está viva, o que morreu foi um serviço. O técnico que montou aquele ponto de presença há quatro anos não trabalha mais na empresa, e a única anotação no inventário diz “servidor Linux”. Antes de tentar qualquer conserto, você digita dois comandos:

```
uname -a
cat /etc/os-release
```

O primeiro responde qual **kernel** está rodando. O segundo responde qual **distribuição** foi instalada. São perguntas diferentes, e tratá-las como se fossem a mesma é a origem de boa parte dos mal-entendidos de quem está começando. Do kernel dependem os drivers, o suporte ao hardware e o comportamento de rede; da distribuição dependem o gerenciador de pacotes, o caminho dos arquivos de configuração, o modelo de atualização e quanto tempo aquela instalação ainda vai receber correções de segurança. Este capítulo desmonta a palavra “Linux” nas partes que ela esconde.

1.1. O que a máquina responde no primeiro minuto

Numa instalação Debian de servidor, os dois comandos acima devolvem algo assim:

```
$ uname -a
Linux pop-torre-03 6.12.13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.13-1
↳ (2025-02-08) x86_64 GNU/Linux
```

```
$ cat /etc/os-release
PRETTY_NAME="Debian GNU/Linux 12 (bookworm)"
NAME="Debian GNU/Linux"
VERSION_ID="12"
VERSION="12 (bookworm)"
ID=debian
HOME_URL="https://www.debian.org/"
```

Repare que os números não conversam: o kernel é 6.12.13, a distribuição é a versão 12. Não há relação alguma entre eles. A mesma máquina, reinstalada com Kali, responderia:

1. O que é Linux: kernel, distribuições e o modelo de software livre

```
$ uname -a
Linux kali 6.12.25-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.12.25-1kali1
↳ (2025-04-30) x86_64 GNU/Linux

$ cat /etc/os-release
PRETTY_NAME="Kali GNU/Linux Rolling"
NAME="Kali GNU/Linux"
VERSION="2025.2"
ID=kali
ID_LIKE=debian
```

Kernel muito parecido, sistema completamente diferente. E o campo `ID_LIKE=debian` já entrega o parentesco: quase tudo que vale para Debian vale para Kali. Guardar essa distinção desde o começo economiza horas de busca em fórum — a resposta que resolve seu problema pode estar escrita para “Linux”, mas depender de um detalhe que só existe no Ubuntu.

1.2. O kernel: o Linux no sentido estrito

Linux, tecnicamente, é só o kernel. É um programa — grande, mas um programa — que fica entre o hardware e todo o resto do software. Quando o computador liga, o firmware carrega o kernel na memória e entrega a ele o controle da máquina; a partir daí é o kernel que decide qual processo usa a CPU e por quanto tempo, quais páginas de memória cada programa enxerga, como os bytes chegam ao disco, como os pacotes saem pela placa de rede e como um dispositivo USB recém-conectado passa a existir para o sistema (Love, 2010).

Nenhum programa comum fala com o hardware diretamente. O `ls` não sabe ler um disco; ele pede ao kernel. O `nmap` não sabe montar um pacote na placa de rede; ele pede ao kernel. Essa fronteira tem nome: **chamada de sistema** (*system call*). O programa roda no que se chama espaço de usuário, com privilégios limitados; o kernel roda em espaço de kernel, com acesso total; e a única porta entre os dois é o conjunto de algumas centenas de chamadas de sistema. Figura 1.1 mostra o arranjo.

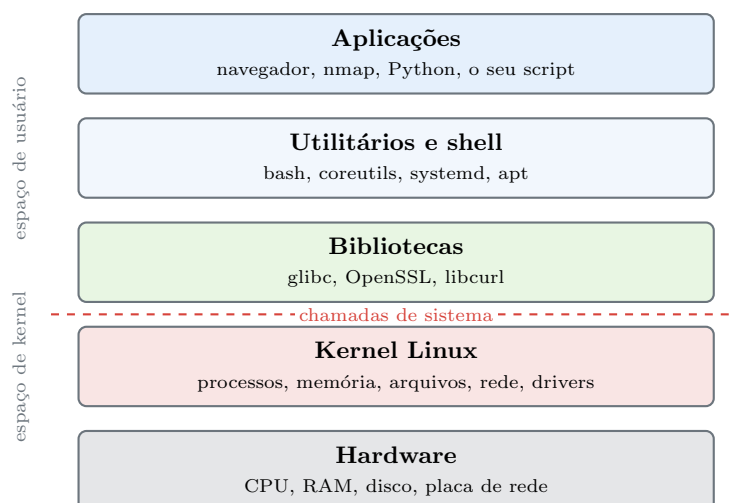


Figura 1.1.: As camadas de um sistema Linux. Tudo acima da linha tracejada precisa pedir ao kernel para tocar no hardware.

Dá para ver essa fronteira funcionando. O comando `strace` intercepta e imprime cada chamada de sistema que um processo faz:

```
strace -c ls /
```

A opção `-c` resume por chamada em vez de listar uma a uma. Um `ls` de um diretório pequeno dispara facilmente umas cem chamadas: `execve` para iniciar o programa, `openat` e `read` para carregar as bibliotecas, `statx` para descobrir o tipo de cada entrada, `getdents64` para ler o diretório, `write` para imprimir na tela. Sem o kernel, o `ls` é um punhado de instruções incapaz de descobrir sequer o próprio nome.

O kernel Linux é **monolítico e modular**. Monolítico porque drivers e subsistemas rodam dentro do mesmo espaço privilegiado, e não como processos separados — decisão de projeto que rendeu, em 1992, um debate público entre Linus Torvalds e Andrew Tanenbaum, autor do Minix, defensor de microkernels. Modular porque partes desse código podem ser carregadas e descarregadas com o sistema em pé:

```
lsmod | head  
modinfo ath9k_htc | head -5
```

O primeiro lista os módulos carregados; o segundo descreve um módulo específico — no exemplo, o driver de um adaptador Wi-Fi USB muito usado em laboratório. É por isso que você pluga um adaptador novo e ele funciona sem reiniciar: o kernel carrega o módulo correspondente sob demanda.

A numeração é simples de ler. Em `6.12.25-amd64`, o `6.12` é a versão de origem (*mainline*), o `25` é o número da correção, e `amd64` é a arquitetura. Algumas versões são marcadas como **LTS** e recebem correções por anos — é o kernel que distribuições de servidor adotam. As versões novas trazem suporte a hardware recente, e é por isso que uma placa de rede lançada este ano pode simplesmente não existir para um servidor com kernel de cinco anos atrás. Esse é o argumento prático mais forte a favor de kernels novos, e o gerenciamento de módulos é tema do Volume 16.

1.3. De onde veio: Unix, GNU e 1991

Em 1969, no Bell Labs da AT&T, Ken Thompson e Dennis Ritchie escreveram o Unix. A virada aconteceu poucos anos depois, quando o sistema foi reescrito em C: um sistema operacional que podia ser recompilado para outra máquina em vez de reescrito em assembly. Junto com o código veio um conjunto de ideias que o Linux herdou por inteiro — tudo é arquivo, programas pequenos que fazem uma coisa bem, saída de um encadeada na entrada do outro, configuração em texto simples (Kernighan; Pike, 1984). Universidades adotaram o Unix, a Universidade da Califórnia em Berkeley criou sua própria linhagem (a família BSD), e nos anos 1980 o código virou produto comercial fechado, com licenças caras e disputas judiciais sobre quem podia distribuir o quê.

Foi contra esse fechamento que Richard Stallman lançou, em 1983, o **Projeto GNU** — sigla recursiva para *GNU's Not Unix*. A meta era um sistema completo, compatível com Unix, cujo código qualquer pessoa pudesse ler, modificar e redistribuir. Em 1991 o projeto tinha praticamente tudo: o compilador `gcc`, o editor Emacs, o `bash`, os utilitários que hoje formam o pacote `coreutils`, o `make`, o depurador. Faltava justamente a peça central — o kernel, chamado Hurd, que se arrastava.

1. O que é Linux: kernel, distribuições e o modelo de software livre

Nesse mesmo ano, um estudante finlandês de 21 anos chamado Linus Torvalds começou a escrever, por conta própria, um kernel para o seu 386. Em agosto de 1991 ele anunciou o projeto numa lista de discussão do Minix com a frase que ficou célebre por envelhecer mal: era só um passatempo, não seria grande e profissional como o GNU. Em 1992 ele relicenciou o kernel sob a **GPL versão 2**, e a peça que faltava ao GNU encontrou o conjunto de ferramentas que faltava ao kernel. O resultado é o que a saída do `uname -a` chama, literalmente, de **GNU/Linux** — e é a razão de existir a disputa sobre esse nome: quem escreve “GNU/Linux” está lembrando que o kernel sozinho não liga uma máquina útil. Neste manual usamos “Linux” no sentido corrente, de sistema completo, e “kernel” quando a distinção importa.

1.4. O que é, então, uma distribuição

Um kernel não abre um terminal. Para ter um sistema operacional utilizável é preciso juntar o kernel a uma biblioteca C, a um programa de inicialização, a um shell, a centenas de utilitários, a um gerenciador de pacotes, a um repositório onde esses pacotes vivem, a um instalador e a um conjunto de decisões padrão. Esse trabalho de integração — escolher versões, aplicar correções, testar combinações, assinar os pacotes e sustentar tudo isso ao longo do tempo — é o que uma **distribuição** faz. Figura 1.2 mostra as peças.

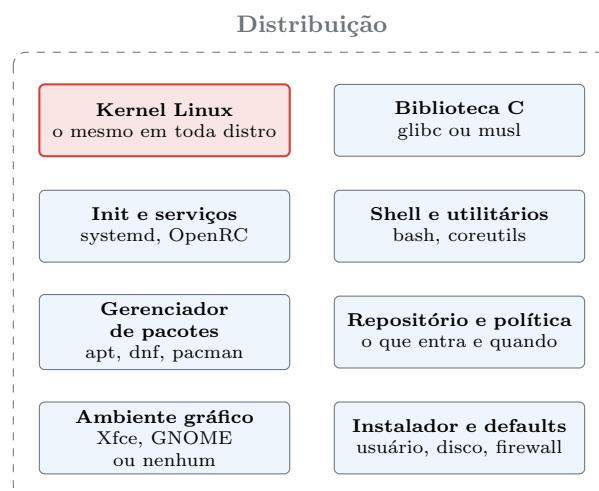


Figura 1.2.: Uma distribuição é o kernel mais o trabalho de integração que o torna utilizável.

Cada uma dessas caixas é uma decisão editorial, e distribuições diferentes decidem diferente. As escolhas que mais mudam o dia a dia são quatro. A primeira é o **modelo de lançamento**: versão fixa, em que os pacotes congelam e só recebem correções de segurança por alguns anos (Debian estável, Ubuntu LTS, RHEL), ou *rolling release*, em que não existe “versão” e o sistema recebe versões novas continuamente (Arch, Kali). A segunda é o **formato de pacote e o gerenciador**: `.deb` com `apt` na família Debian, `.rpm` com `dnf` na família Red Hat, e por aí vai. A terceira é o **conjunto padrão**: uma imagem mínima de servidor, um desktop completo, ou — no caso do Kali — algumas centenas de ferramentas de segurança já instaladas. A quarta é a **política de segurança e suporte**: quem corrige, com que rapidez, e por quanto tempo.

Para ter uma ideia do tamanho do trabalho, conte os pacotes de uma instalação comum:

```
dpkg -l | grep -c '^ii'
```


Uma instalação enxuta de Debian passa dos 500 pacotes; um Kali com o metapacote padrão passa fácil dos 3.000. Nenhum deles foi escrito pela distribuição — todos foram empacotados, testados em conjunto e assinados por ela.

1.5. Famílias: o mapa das distribuições

Existem centenas de distribuições, mas quase todas descendem de meia dúzia de troncos, e é o tronco que determina os comandos que você vai digitar. Figura 1.3 resume as linhagens que importam na prática.

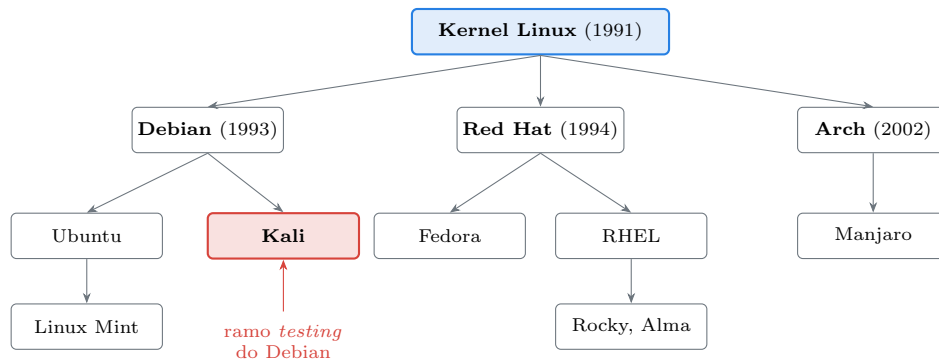


Figura 1.3.: As linhagens principais. O tronco define o gerenciador de pacotes e boa parte das convenções.

A **família Debian** usa pacotes `.deb` e o `apt`. Debian é a base; Ubuntu deriva dela e adiciona um ciclo de lançamento previsível; Kali deriva do ramo de testes do Debian; Linux Mint deriva do Ubuntu; o Raspberry Pi OS também é Debian. É a família com maior presença em servidores de pequeno e médio porte no Brasil, e é a família deste manual.

A **família Red Hat** usa pacotes `.rpm` e o `dnf`. Fedora é o laboratório onde as novidades aparecem primeiro; o RHEL é o produto comercial estabilizado a partir dele; Rocky Linux e AlmaLinux são reconstruções gratuitas e compatíveis do RHEL. É a família dominante em datacenter corporativo, e traz consigo `firewalld` e SELinux, que resolvem os mesmos problemas de outro jeito.

A **família Arch** é *rolling*, minimalista e monta o sistema a partir do que o usuário escolhe instalar. Fora desses três troncos, valem menção o openSUSE, o Alpine — minúsculo, com biblioteca C `musl` no lugar da `glibc`, quase padrão em contêineres — e o Gentoo, que compila tudo a partir do fonte.

A consequência prática é direta: **os conceitos são os mesmos, os comandos nem sempre**. Processo, permissão, descritor de arquivo, socket, sinal — isso é kernel, e é idêntico em toda parte. Já `apt install` contra `dnf install`, `/etc/network/interfaces` contra as conexões do NetworkManager, `nftables` contra `firewalld`, isso é distribuição. Ao escolher a distribuição de um servidor que vai ficar quatro anos num rack de POP, o critério raramente é gosto: é janela de suporte, previsibilidade de atualização e disponibilidade dos pacotes de que você depende (Nemeth et al., 2017).

1.6. O modelo de software livre

Nada disso seria possível sem o arranjo jurídico que sustenta o ecossistema. Software livre, na definição da Free Software Foundation, é o que garante quatro liberdades ao usuário: **liberdade 0**, executar o programa para qualquer finalidade; **liberdade 1**, estudar como ele funciona e adaptá-lo, o que exige acesso ao código-fonte; **liberdade 2**, redistribuir cópias; e **liberdade 3**, distribuir versões modificadas. “Livre” aqui é liberdade, não prego — a ambiguidade existe em inglês (*free*) e por isso surgiu o termo alternativo *open source*, cunhado em 1998 com ênfase mais pragmática, no modelo de desenvolvimento colaborativo, e menos na dimensão ética. Na prática as duas listas de licenças aprovadas quase coincidem.

As licenças se dividem em duas famílias. As **copyleft**, como a GPL, obrigam quem distribui uma versão modificada a distribuí-la sob a mesma licença, com o fonte disponível — a liberdade é passada adiante à força. As **permissivas**, como MIT, BSD e Apache, permitem que o código seja incorporado a produtos fechados. O kernel Linux é GPLv2. É por isso que um fabricante que embarca Linux num roteador ou numa OLT precisa disponibilizar o fonte do kernel modificado que usou — e é por isso que existe, para muito equipamento de rede, uma comunidade capaz de compilar firmware alternativo.

Esse mesmo mecanismo é o que permite a existência das distribuições: o Kali pode pegar o Debian inteiro, modificar, rebatizar e redistribuir porque as licenças autorizam. Cada pacote instalado traz sua licença registrada em disco:

```
head -20 /usr/share/doc/coreutils/copyright
apt show nmap | head -15
```

Vale desfazer um mal-entendido comum: software livre não é software sem dinheiro envolvido. Red Hat, Canonical e SUSE são empresas grandes que vendem suporte, certificação e serviços em cima de código aberto; a OffSec, que mantém o Kali, vende treinamento e certificação; a maior parte dos commits do kernel Linux vem de gente paga para isso, empregada por fabricantes de hardware e provedores de nuvem. O que o modelo elimina não é o pagamento — é o aprisionamento.

1.7. O que muda no seu dia a dia

Três consequências concretas. Primeira: quando um serviço se comporta de um jeito que a documentação não explica, o código está a um `apt source` de distância, e ler o fonte é uma opção real de diagnóstico, não um recurso teórico. Segunda: o repositório da distribuição é a sua cadeia de suprimentos de software, com pacotes assinados e verificados a cada `apt update` — instalar `.deb` baixado de qualquer lugar contorna essa verificação, e o assunto volta com força no Volume 6. Terceira: você pode auditar. Num ambiente onde a máquina fica trancada num contêiner metálico no meio de um pasto, saber exatamente o que está rodando e de onde veio cada binário não é preciosismo — é a diferença entre um incidente investigável e um mistério.

1.8. No Kali

O Kali Linux é uma distribuição **Debian rolling** mantida pela OffSec, montada a partir do ramo `testing` do Debian e voltada a teste de invasão, forense e auditoria de segurança (Offensive Security, 2025). Alguns pontos em que ele difere de um Debian ou Ubuntu comum e que aparecem já no primeiro contato:

- **Não existe “versão” no sentido do Debian.** O 2025.2 de `/etc/os-release` é um marco de imagem de instalação, não um congelamento: a atualização vem do ramo `kali-rolling`, que se atualiza continuamente. Confira com `cat /etc/apt/sources.list` — a linha aponta para `kali-rolling`, não para um codinome como `bookworm`. Existem ainda os ramos `kali-last-snapshot`, congelado no último lançamento, e `kali-experimental`. Isso significa que `apt update && apt full-upgrade` no Kali é uma operação de rotina, não um evento raro.
- **O usuário padrão é kali, não root.** Até a versão 2020.1 o Kali logava como `root` por padrão; hoje adota o modelo de usuário comum com `sudo`, como qualquer Debian. O tema é aprofundado no Volume 4.
- **O kernel tem patches próprios.** O sufixo `-kali1` em `uname -r` indica um kernel com correções aplicadas pela distribuição, notadamente para injeção de pacotes e modo monitor em adaptadores Wi-Fi — recursos que um kernel Debian puro nem sempre oferece.
- **As ferramentas vêm por metapacotes.** `kali-linux-default` traz o conjunto usual, `kali-linux-large` traz quase tudo, `kali-linux-headless` traz só o que roda sem interface gráfica. Instalar um metapacote arrasta centenas de programas de uma vez. Detalhes no Volume 6.
- **kali-tweaks** é um utilitário próprio da distribuição para ajustar rapidamente ramo de repositório, shell padrão, serviços de rede e configurações de hardening.
- **Kali não é distribuição de servidor.** Ela vem com ferramentas ofensivas instaladas, acompanha um ramo de testes e não tem janela de suporte de longo prazo. Para o servidor de monitoramento da história de abertura, a escolha correta é Debian estável ou Ubuntu LTS. Kali é a máquina de trabalho de quem audita, não a máquina auditada.

Nota sobre a família RHEL. Em Fedora, RHEL, Rocky ou Alma, os equivalentes são `cat /etc/redhat-release` no lugar do `os-release`, `dnf` no lugar do `apt`, `rpm -qa` no lugar do `dpkg -l`, `firewalld` no lugar do `nftables` direto e SELinux no lugar do AppArmor. O kernel e os conceitos são os mesmos.

1.9. Laboratório

Roteiro para executar numa máquina virtual com Kali. Nenhum passo altera o sistema; todos são de leitura.

1. Identifique kernel e distribuição separadamente.

```
uname -r
uname -a
cat /etc/os-release
hostnamectl
```

Esperado: `uname -r` devolve algo como `6.12.25-amd64`; o `os-release` devolve `ID=kali` e `ID_LIKE=debian`. O `hostnamectl` reúne as duas informações em campos separados — Operating System e Kernel. Anote os dois valores; eles vão divergir na próxima atualização, e em ritmos diferentes.

2. Confirme que o kernel é modular.

```
lsmod | wc -l
lsmod | head -5
```

1. O que é Linux: kernel, distribuições e o modelo de software livre

Esperado: dezenas de módulos carregados. A primeira coluna é o nome, a terceira é a contagem de uso — módulo com contagem zero não está sendo usado por ninguém no momento.

3. Observe a fronteira das chamadas de sistema.

```
sudo apt install -y strace
strace -c ls / 2>&1 | tail -20
```

Esperado: uma tabela com as chamadas usadas e quantas vezes cada uma. Procure `openat`, `read`, `write`, `getdents64`, `execve`. A conclusão a tirar: um comando trivial como `ls` não faz nada sozinho — ele passa todo o trabalho real para o kernel.

4. Veja que o programa depende de bibliotecas, e não só do kernel.

```
ldd /bin/ls
/lib/x86_64-linux-gnu/libc.so.6 --version | head -2
```

Esperado: a lista de bibliotecas compartilhadas de que o `ls` precisa, incluindo `libc.so.6`, e a versão da glibc. Essa camada pertence à distribuição, não ao kernel — trocar de distribuição pode trocar a glibc; trocar de kernel, não.

5. Meça o trabalho de integração da distribuição.

```
dpkg -l | grep -c '^ii'
apt list --installed 2>/dev/null | wc -l
```

Esperado: alguns milhares de pacotes num Kali com metapacote padrão. Cada um foi empacotado, testado em conjunto e assinado por alguém.

6. Leia a licença de um programa que você usa.

```
head -25 /usr/share/doc/nmap/copyright
apt show bash 2>/dev/null | head -12
```

Esperado: o arquivo `copyright` traz autoria e licença. Compare a licença do `bash` (GPL) com a de algum utilitário sob MIT ou BSD e observe a diferença de exigência sobre versões modificadas.

7. Descubra o ramo de repositório em que a sua máquina está.

```
cat /etc/apt/sources.list
apt policy
```

Esperado: uma linha apontando para `kali-rolling`. Se apontar para `kali-last-snapshot`, sua máquina está congelada no último lançamento e não receberá pacotes novos até você mudar isso.

8. A prova final de que kernel e distribuição são coisas distintas. Se houver Docker no ambiente:

```
sudo docker run --rm fedora cat /etc/os-release | head -3
sudo docker run --rm fedora uname -r
uname -r
```

Esperado: o `os-release` de dentro do contêiner diz Fedora; o `uname -r` de dentro do contêiner devolve **exatamente o mesmo kernel** da sua máquina Kali. O contêiner traz uma distribuição inteira — utilitários, `dnf`, bibliotecas — mas não traz kernel: ele usa o do hospedeiro. É a demonstração mais limpa da separação, e o mecanismo por trás dela é assunto do Volume 16.

1.10. Resumo

- **Linux, no sentido estrito, é o kernel:** o programa que gerencia CPU, memória, arquivos, rede e dispositivos, e que expõe tudo isso ao restante do software por meio das chamadas de sistema.
- **Distribuição é o kernel mais o trabalho de integração:** biblioteca C, init, shell, utilitários, gerenciador de pacotes, repositório, instalador e uma política de atualização e suporte.
- `uname -a` e `/etc/os-release` respondem perguntas diferentes e seus números não têm relação entre si. Saber ler os dois é o primeiro diagnóstico de qualquer máquina desconhecida.
- **A família da distribuição define os comandos.** Conceitos de kernel valem em toda parte; `apt`, `dnf` e `pacman` não. Debian e derivados — Kali incluído — formam a família deste manual.
- **O modelo de software livre é o que torna esse ecossistema possível:** quatro liberdades, licenças copyleft como a GPLv2 do kernel e permissivas como MIT e BSD, e um mercado que cobra por suporte e serviço, não por aprisionamento.
- **Kali é Debian rolling voltado à segurança,** com usuário `kali`, kernel com patches próprios, ferramentas em metapacotes e sem janela de suporte longo — excelente estação de trabalho de auditoria, má escolha para servidor de produção.

2. Por que Kali Linux: propósito, filosofia e quando não usá-lo

(em elaboracao)

3. Instalando o Kali: máquina virtual, bare metal, WSL e live USB

(em elaboracao)

4. Primeiro contato: sessão, ambiente Xfce e organização do sistema

(em elaboracao)

5. A estrutura de diretórios do Linux (FHS)

(em elaboracao)

6. Anatomia de um comando: shell, argumentos, opções e sintaxe

(em elaboracao)

7. Obtendo ajuda: man, info, tldr e a documentação do Kali

(em elaboracao)

Parte II.

Volume 2 — O Terminal e o Shell

8. Emulador de terminal e shell: bash, zsh e o padrão do Kali

(em elaboracao)

9. Navegação: pwd, cd, ls e caminhos absolutos e relativos

(em elaboracao)

10. Manipulando arquivos e diretórios: cp, mv, rm, mkdir

(em elaboracao)

11. Visualizando conteúdo: cat, less, head, tail e more

(em elaboracao)

12. Redirecionamento e pipes: stdin, stdout e stderr

(em elaboracao)

13. Curingas, globbing e expansão de chaves

(em elaboracao)

14. Histórico, atalhos de teclado e produtividade no terminal

(em elaboracao)

Parte III.

**Volume 3 — Arquivos e o Sistema de
Arquivos**

15. Tipos de arquivo, inodes e a árvore de diretórios

(em elaboracao)

16. Links simbólicos e hard links

(em elaboracao)

17. Localizando arquivos: find, locate, which e type

(em elaboracao)

18. Compactação e arquivamento: tar, gzip, xz e zip

(em elaboracao)

19. Montagem de dispositivos e o comando mount

(em elaboracao)

20. Permissões de arquivo: leitura, escrita e execução

(em elaboracao)

Parte IV.

**Volume 4 — Usuários, Grupos e
Permissões**

21. Usuários, grupos e o arquivo /etc/passwd

(em elaboracao)

22. O root, sudo e a elevação de privilégios

(em elaboracao)

23. Permissões especiais: SUID, SGID e sticky bit

(em elaboracao)

24. ACLs e atributos estendidos de arquivo

(em elaboracao)

25. Gerenciamento de usuários, senhas e o PAM

(em elaboracao)

26. Trabalhando como root no Kali com segurança

(em elaboracao)

Parte V.

**Volume 5 — Processos e Controle de
Tarefas**

27. O que é um processo: PID, estados e a árvore de processos

(em elaboracao)

28. Monitoramento: ps, top e htop

(em elaboracao)

29. Sinais e controle: kill, killall, nice e prioridades

(em elaboracao)

30. Jobs, primeiro e segundo plano, nohup e disown

(em elaboracao)

31. Multiplexação de terminal: tmux e screen

(em elaboracao)

32. Recursos do sistema: memória, CPU, uptime e limites

(em elaboracao)

Parte VI.

Volume 6 — Gerenciamento de Pacotes

33. APT e o modelo Debian: repositórios e o Kali rolling

(em elaboracao)

34. Instalando, removendo e atualizando pacotes com apt

(em elaboracao)

35. dpkg, arquivos .deb e resolução de dependências

(em elaboracao)

36. Repositórios do Kali, metapacotes e kali-tweaks

(em elaboracao)

37. Compilando a partir do código-fonte

(em elaboracao)

38. Alternativas: pipx, snap, flatpak e AppImage

(em elaboracao)

Parte VII.

Volume 7 — Boot, systemd e Serviços

39. O processo de inicialização: UEFI, GRUB e o kernel

(em elaboracao)

40. systemd: units, targets e o modelo de serviços

(em elaboracao)

41. Gerenciando serviços com systemctl

(em elaboracao)

42. Logs com journald e o syslog tradicional

(em elaboracao)

43. Agendamento: cron, at e timers do systemd

(em elaboracao)

44. Targets, modo de recuperação e resolução de problemas de boot

(em elaboracao)

Parte VIII.

**Volume 8 — Armazenamento e
Sistemas de Arquivos**

45. Discos, partições e a nomenclatura de dispositivos

(em elaboracao)

46. Particionamento com fdisk, parted e gdisk

(em elaboracao)

47. Sistemas de arquivos: ext4, Btrfs, XFS e formatação

(em elaboracao)

48. LVM: volumes lógicos flexíveis

(em elaboracao)

49. RAID por software com mdadm

(em elaboracao)

50. Criptografia de disco com LUKS e a persistência no Kali

(em elaboracao)

Parte IX.

**Volume 9 — Fundamentos de Shell
Scripting**

51. Do comando ao script: o primeiro script Bash

(em elaboracao)

52. Variáveis, o ambiente e o escopo de exportação

(em elaboracao)

53. Aspas, expansões e substituição de comandos

(em elaboracao)

54. Entrada e saída: read, echo e printf

(em elaboracao)

55. Condicionais: test, colchetes duplos e if

(em elaboracao)

56. Códigos de saída e o encadeamento de comandos

(em elaboracao)

57. Boas práticas: shebang, permissões e portabilidade

(em elaboracao)

Parte X.

Volume 10 — Scripting Intermediário

58. Laços: for, while e until

(em elaboracao)

59. Funções, retorno e escopo de variáveis

(em elaboracao)

60. Arrays indexados e associativos

(em elaboracao)

61. Parâmetros posicionais e getopt

(em elaboracao)

62. Expansão de parâmetros avançada

(em elaboracao)

63. Aritmética e manipulação de números no shell

(em elaboracao)

Parte XI.

Volume 11 — Processamento de Texto

64. Expressões regulares: fundamentos e os diferentes sabores

(em elaboracao)

65. grep e a busca de padrões

(em elaboracao)

66. sed: o editor de fluxo

(em elaboracao)

67. awk: processamento por campos e registros

(em elaboracao)

68. cut, sort, uniq, tr e a caixa de ferramentas de texto

(em elaboracao)

69. Dados estruturados na linha de comando: jq, JSON e CSV

(em elaboracao)

70. Juntando tudo: pipelines de processamento de dados

(em elaboracao)

Parte XII.

**Volume 12 — Scripting Avançado e
Robusto**

71. Tratamento de erros: set -euo pipefail e traps

(em elaboracao)

72. Depuração de scripts: set -x e ShellCheck

(em elaboracao)

73. Manipulando arquivos, descritores e processos em scripts

(em elaboracao)

74. Subshells, agrupamento e here-documents

(em elaboracao)

75. Sinais, processos filhos e execução em paralelo

(em elaboracao)

76. Interação com o usuário: menus, cores e caixas de diálogo

(em elaboracao)

77. Estruturando scripts grandes e bibliotecas de funções

(em elaboracao)

Parte XIII.

Volume 13 — Redes no Linux

78. Fundamentos: interfaces, endereçamento IP e a pilha de rede

(em elaboracao)

79. Configuração de rede: ip, NetworkManager e arquivos

(em elaboracao)

80. Diagnóstico: ping, traceroute, ss, dig e mtr

(em elaboracao)

81. Acesso e transferência remota: SSH, scp e rsync

(em elaboracao)

82. HTTP na linha de comando: curl, wget e netcat

(em elaboracao)

83. Firewall no Linux: nftables e iptables

(em elaboracao)

84. Captura e análise de tráfego: tcpdump e tshark

(em elaboracao)

Parte XIV.

**Volume 14 — Segurança e o Ambiente
Kali**

85. Hardening do Linux: superfície de ataque e princípios

(em elaboracao)

86. Gerenciamento de segredos: chaves SSH e GPG

(em elaboracao)

87. Capabilities, isolamento e menor privilégio

(em elaboracao)

88. Confinamento: AppArmor, SELinux e sandboxing

(em elaboracao)

89. Auditoria, integridade de arquivos e detecção de alterações

(em elaboracao)

90. O ecossistema de ferramentas do Kali: categorias e fluxos

(em elaboracao)

91. Anonimato e privacidade: proxychains, Tor e VPN

(em elaboracao)

Parte XV.

**Volume 15 — Automação e
Produtividade**

92. Personalizando o shell: .bashrc, .zshrc, aliases e funções

(em elaboracao)

93. Zsh, plugins e o prompt do Kali

(em elaboracao)

94. Automatizando tarefas com cron e scripts agendados

(em elaboracao)

95. Controle de versão com Git na linha de comando

(em elaboracao)

96. Editores no terminal: vim e nano

(em elaboracao)

97. Dotfiles e ambientes reproduzíveis

(em elaboracao)

Parte XVI.

**Volume 16 — Kernel, Contêineres e
Virtualização**

98. O kernel Linux: módulos, /proc e /sys

(em elaboracao)

99. Contêineres por dentro: namespaces, cgroups e Docker

(em elaboracao)

100. Virtualização: KVM, QEMU e o laboratório de estudos

(em elaboracao)

101. Compilação, toolchains e o ecossistema de desenvolvimento

(em elaboracao)

102. Observabilidade e desempenho: strace, ltrace e perf

(em elaboracao)

103. Para onde ir depois: certificações, comunidade e prática contínua

(em elaboracao)

Referências

KERNIGHAN, Brian W.; PIKE, Rob. **The Unix Programming Environment**. Englewood Cliffs, NJ: Prentice Hall, 1984.

LOVE, Robert. **Linux Kernel Development**. 3. ed. Upper Saddle River, NJ: Addison-Wesley, 2010.

NEMETH, Evi *et al.* **UNIX and Linux System Administration Handbook**. 5. ed. Boston, MA: Addison-Wesley, 2017.

OFFENSIVE SECURITY. **Kali Linux Documentation**. <https://www.kali.org/docs/>, 2025.

